📘 Introduction to Web API in .NET

A **Web API** (Application Programming Interface) is a framework that allows you to expose business logic and data to clients over HTTP. It is a foundational component of modern web development, enabling communication between server-side applications and client-side consumers like web apps, mobile apps, or other services.

In the .NET ecosystem, Web API is part of **ASP.NET Core** and is used to build RESTful services. With ASP.NET Core, APIs are lightweight, high-performance, and cross-platform.

🧩 Why Use Web API?

- Enables **cross-platform communication** using HTTP/HTTPS
- Easily consumed by any client that understands HTTP (Angular, React, mobile apps)
- Lightweight and ideal for **microservices architecture**
- Supports **JSON/XML** formats
- Integrates easily with **Swagger** for testing and documentation
- Allows **token-based authentication** using **JWT (JSON Web Token)**

🏗️ Web API Architecture Overview

A Web API application typically follows a **layered architecture** which ensures separation of concerns and better maintainability:

1. Controller Layer

- Acts as the **entry point** for incoming HTTP requests
- Maps routes to methods (`[HttpGet]`, `[HttpPost]`, etc.)
- Validates the input and delegates business logic to the service layer

2. Service Layer (Business Logic)

- Contains core application logic (e.g., generating JWT tokens, calculating totals, validating users)
- Keeps controller clean and focused on handling HTTP aspects

3. Data Access Layer (optional for DB)

- Handles data retrieval or storage (e.g., using Entity Framework to talk to SQL Server)
- Isolates data-related code from business logic

4. Middleware Pipeline

- ASP.NET Core processes HTTP requests using a **middleware pipeline**
- Each middleware performs specific tasks like routing, authentication, exception handling, etc.

5. Security Layer – JWT Authentication

- A **JWT (JSON Web Token)** is issued when a user logs in successfully
- The token is attached in the **Authorization header** of every request
- Secured endpoints validate the token before processing the request

---

🔐 How JWT Authentication Works in Web API

Here's how the typical **JWT authentication flow** works in a Web API project:

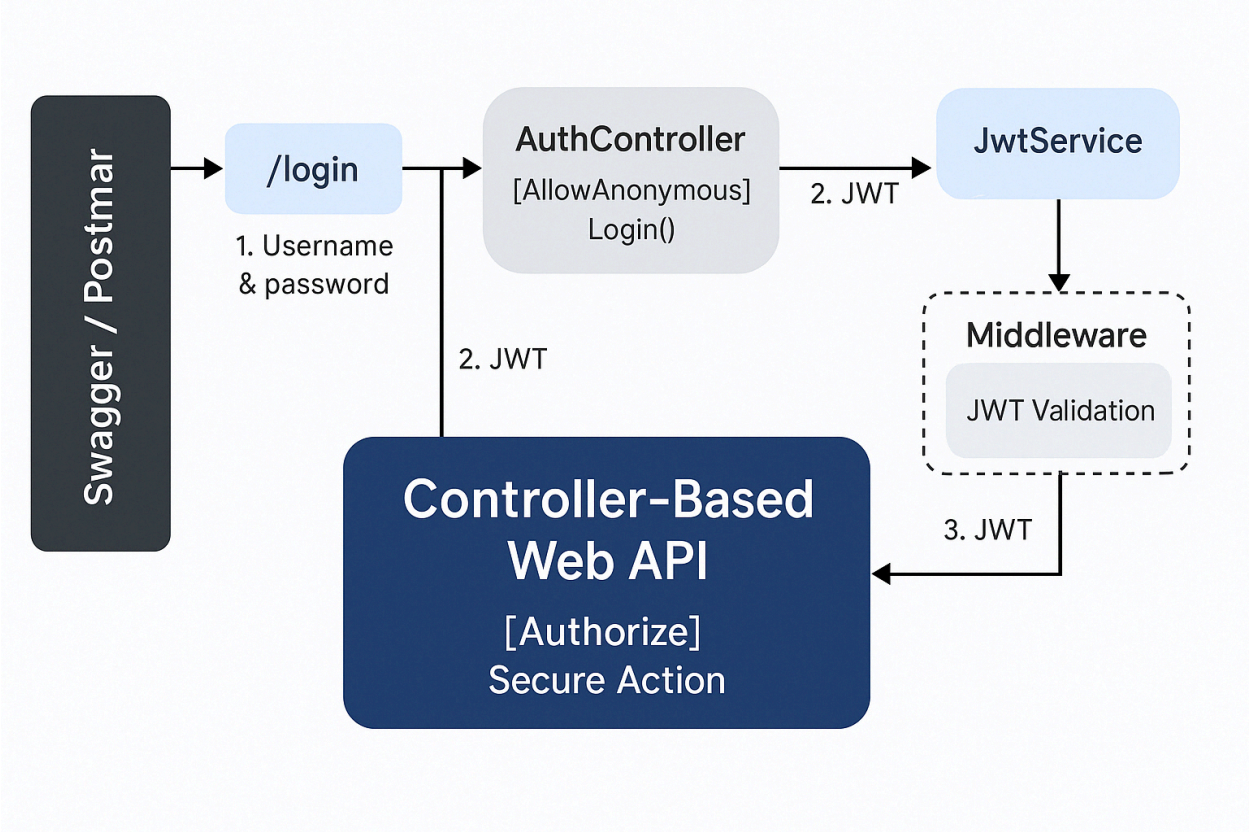
1. **User sends credentials** to `/login` endpoint.
2. **Web API validates credentials** and generates a JWT token.
3. Token is **returned to the client**.

4. Client sends **token in Authorization header** (`Bearer <token>`) with every secured request.
5. Web API **verifies token** using `JwtBearer` middleware.
6. If valid, request is allowed to access the protected endpoint.

Architecture

- Request flow from Swagger/Postman to `/login`
- JWT generation in `JwtService`
- Controller with `[Authorize]`
- JWT validation in Middleware pipeline

Architecture Diagram



✅ Step-by-Step Explanation: JWT Authentication in .NET 8 Minimal API

🔷 1. Project Overview

This application demonstrates:

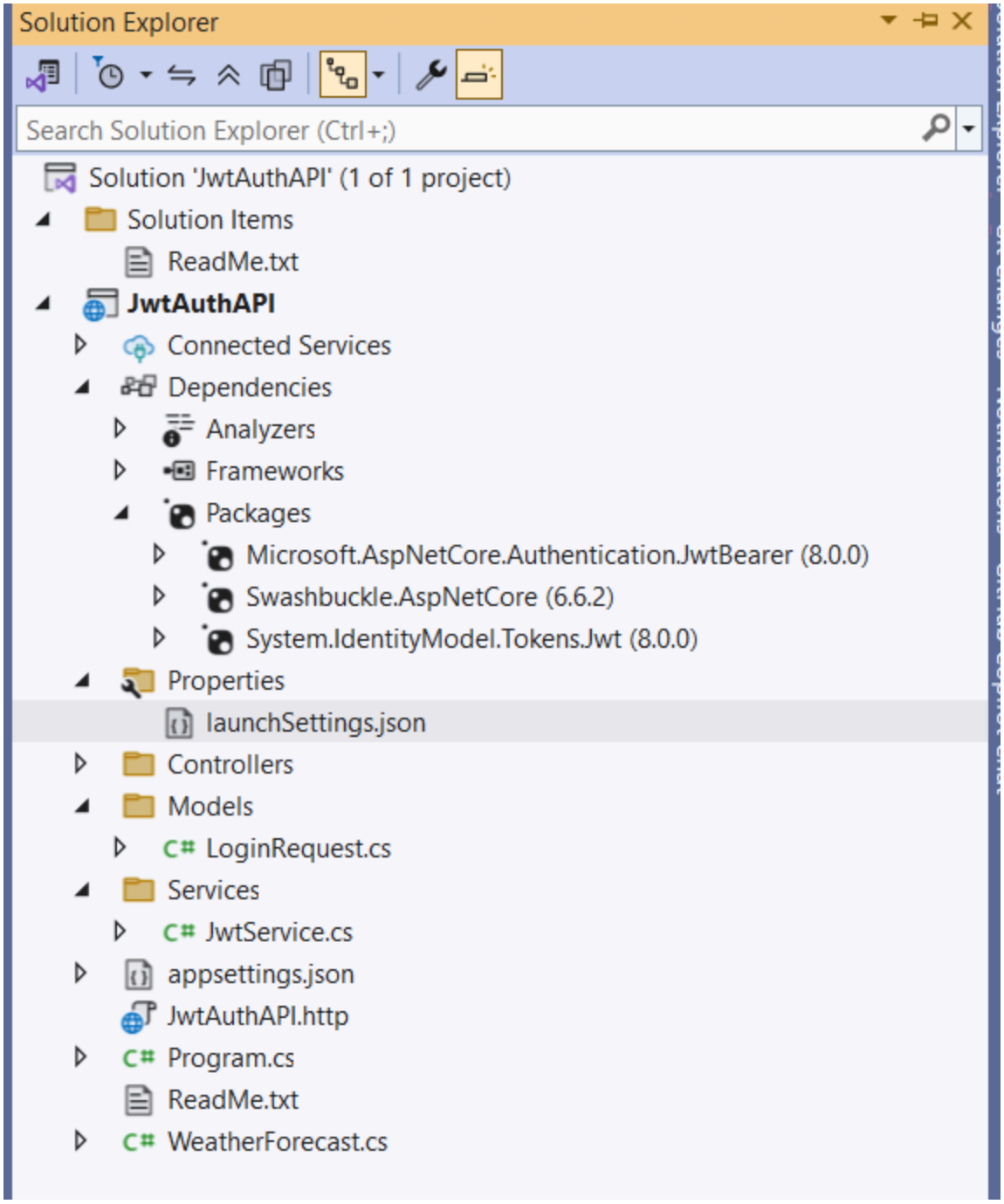
- JWT Authentication in a .NET 8 Minimal API.
- Swagger integration to test secured endpoints.
- Use of `Program.cs` only (no Controllers or Startup.cs).
- Login endpoint to generate JWT tokens.
- Secure endpoint accessible only with valid JWT.

🔷 2. Folder & File Structure

```
JwtAuthAPI/
│
├── Models/
│   └── LoginRequest.cs
│
├── Services/
│   └── JwtService.cs
│
├── appsettings.json
├── Program.cs
```

Packages to be installed

1. `Microsoft.AspNetCore.Authentication.JwtBearer ( stable version compatible to .NET 8)`
2. `System.IdentityModel.Tokens.Jwt ( stable version compatible to .NET 8`)


Solution Explorer
Search Solution Explorer (Ctrl+;)
Solution 'JwtAuthAPI' (1 of 1 project)
Solution Items
ReadMe.txt
JwtAuthAPI
Connected Services
Dependencies
Analyzers
Frameworks
Packages
Microsoft.AspNetCore.Authentication.JwtBearer (8.0.0)
Swashbuckle.AspNetCore (6.6.2)
System.IdentityModel.Tokens.Jwt (8.0.0)
Properties
launchSettings.json
Controllers
Models
LoginRequest.cs
Services
JwtService.cs
appsettings.json
JwtAuthAPI.http
Program.cs
ReadMe.txt
WeatherForecast.cs

🔷 3. appsettings.json (for JWT configuration)

```
{
  "Jwt": {
    "Key": "xxxxxxxx",
    "Issuer": "https://localhost:5000",
    "Audience": "https://localhost:5000",
    "ExpiryInMinutes": 60
  },
  ...
}
```

✅ **Purpose**: Stores the symmetric key and token metadata. The key can be generated using a C# console application. Othewerise , we can generate the key using the below command

Developer Powershell

$bytes = New-Object Byte[] 32; [System.Security.Cryptography.RandomNumberGenerator]::Create().GetBytes($bytes); [Convert]::ToBase64String($bytes)

How to add issuer and the Audience url in appsettings.json

Navigate to **Properties-> LaunchSettings.json;** Use the **applicationUrl** under **https** section as shown below

```
"https": {
  "commandName": "Project",
  "dotnetRunMessages": true,
  "launchBrowser": true,
  "launchUrl": "swagger",
  "applicationUrl": "https://localhost:xxxx;http://localhost:xxx1",
  "environmentVariables": {
    "ASPNETCORE_ENVIRONMENT": "Development"
  }
```

🔷 4. Model – `LoginRequest.cs`

```
public class LoginRequest
{
    public string Username { get; set; }
    public string Password { get; set; }
}
```

✅ **Purpose**: Represents the body of the login request (`/login` endpoint).

🔷 5. JWT Service – `JwtService.cs`

```
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;
using Microsoft.IdentityModel.Tokens;

namespace JwtAuthAPI.Services
{
    public class JwtService
    {
        private readonly string _key;
        private readonly string _issuer;
        private readonly string _audience;

        public JwtService(IConfiguration configuration)
        {
            _key = configuration["Jwt:Key"];
            _issuer = configuration["Jwt:Issuer"];
            _audience = configuration["Jwt:Audience"];
        }

        public string GenerateToken(string username)
        {
            var tokenHandler = new JwtSecurityTokenHandler();
```

```
            var key = Encoding.UTF8.GetBytes(_key);

            var tokenDescriptor = new SecurityTokenDescriptor
            {
                Subject = new ClaimsIdentity(new[] {
                    new Claim(ClaimTypes.Name, username)
                }),
                Expires = DateTime.UtcNow.AddMinutes(60),
                Issuer = _issuer,
                Audience = _audience,
                       SigningCredentials = new SigningCredentials(new
SymmetricSecurityKey(key), SecurityAlgorithms.HmacSha256)
            };

            var token = tokenHandler.CreateToken(tokenDescriptor);
            return tokenHandler.WriteToken(token);
        }
    }
}
```

✅ **Purpose**: Generates JWT with claims, expiry, signing key, issuer & audience.

🔷 6. `Program.cs` – The Main Configuration

🛠️ Services Setup

```
builder.Services.AddSingleton<JwtService>();

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationSch
eme)
    .AddJwtBearer(options =>
    {
                                                var      key      =
Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]);
                            options.TokenValidationParameters    =    new
TokenValidationParameters
```

```
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = builder.Configuration["Jwt:Issuer"],
            ValidAudience = builder.Configuration["Jwt:Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(key)
        };
    });

builder.Services.AddAuthorization();
```

> ✅ Enables JWT Bearer authentication & validation of token signature, issuer, and audience.

💬 Swagger with JWT Support

```
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(options =>
{
    options.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
    {
        Name = "Authorization",
        Type = SecuritySchemeType.ApiKey,
        Scheme = "Bearer",
        BearerFormat = "JWT",
        In = ParameterLocation.Header,
         Description = "Enter 'Bearer' followed by a space and your JWT
token."
    });

    options.AddSecurityRequirement(new OpenApiSecurityRequirement
    {
        {
            new OpenApiSecurityScheme
            {
```

```
                Reference = new OpenApiReference
                {
                    Type = ReferenceType.SecurityScheme,
                    Id = "Bearer"
                }
            },
            Array.Empty<string>()
        }
    });
});
```

> ✅ Swagger supports JWT so you can test secure endpoints by providing a token in the **Authorize** popup.

🚀 Middleware Pipeline

```
var app = builder.Build();

app.UseAuthentication();
app.UseAuthorization();

if (app.Environment.IsDevelopment())
{
    app.UseDeveloperExceptionPage();
    app.UseSwagger();
    app.UseSwaggerUI();
}
```

> ✅ Adds middleware to authenticate and authorize requests. Swagger is enabled only in development.

🔐 Login & Secure API Endpoints

```
app.MapPost("/login", (LoginRequest request, JwtService jwtService) =>
{
    if (request.Username == "admin" && request.Password == "password")
    {
        var token = jwtService.GenerateToken(request.Username);
        return Results.Ok(new { Token = token });
    }
    return Results.Unauthorized();
});

app.MapGet("/secure-data", (HttpContext context) =>
{
    var username = context.User.Identity?.Name ?? "Unknown";
    return Results.Ok($"This is secured data for {username}");
})
.RequireAuthorization();
```

🔑 `/login`: Authenticates hardcoded credentials and returns a JWT token
🔐 `/secure-data`: Protected endpoint requiring a valid JWT token

🔷 7. Testing in Swagger

✅ Step-by-step:

1. **Run the app**
   Visit: `https://localhost:{port}/swagger`

Expand `/login` → Try it out
Provide:

```
{
  "username": "admin",
  "password": "password"
}
```

2. Click **Execute**

Copy token from response
Looks like:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9…
```

3. **Click "Authorize" button at the top**
Enter:

```
Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

4. **Call `/secure-data` → Should return authorized response**

🧠 Common Mistakes to Avoid in Swagger

Mistake	Solution
Missing "Bearer " prefix in token	Always prefix token in Swagger with `Bearer`
Invalid audience/issuer	Make sure values match between token & appsettings
Expired token	Check expiry, regenerate token
`UseAuthentication()` missing	Ensure it's before `UseAuthorization()` in middleware

Controller-based API

Extending the minimal API into a **controller-based API** is a valuable learning progression at the next level. It introduces the MVC architecture and prepares them for real-world .NET Core development, especially when working on large projects.

✅ Goal:

Migrate your **Minimal API (`Program.cs`)** implementation of **JWT-based authentication** to a **Controller-based Web API**, using:

- Controller for login
- Controller for secure endpoints
- Proper routing and attribute-based authorization

🔧 Step-by-Step Refactor to Controller-Based JWT Auth API:

---

1. Models

Your existing `LoginRequest` model is fine:

📁 Models/LoginRequest.cs

```
namespace JwtAuthAPI.Models
{
    public class LoginRequest
    {
        public string Username { get; set; }
        public string Password { get; set; }
    }
}
```

---

2. Services

Your `JwtService` stays the same.

📁 **Services/JwtService.cs**

```
using Microsoft.Extensions.Configuration;
using Microsoft.IdentityModel.Tokens;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;

namespace JwtAuthAPI.Services
{
    public class JwtService
    {
        private readonly string _key;
        private readonly string _issuer;
        private readonly string _audience;

        public JwtService(IConfiguration configuration)
        {
            _key = configuration["Jwt:Key"];
            _issuer = configuration["Jwt:Issuer"];
            _audience = configuration["Jwt:Audience"];
        }

        public string GenerateToken(string username)
        {
            var tokenHandler = new JwtSecurityTokenHandler();
            var key = Encoding.UTF8.GetBytes(_key);

            var tokenDescriptor = new SecurityTokenDescriptor
            {
                Subject = new ClaimsIdentity(new[]
                {
                    new Claim(ClaimTypes.Name, username)
                }),
```

```
                Expires = DateTime.UtcNow.AddMinutes(60),
                Issuer = _issuer,
                Audience = _audience,
                       SigningCredentials = new SigningCredentials(new
SymmetricSecurityKey(key), SecurityAlgorithms.HmacSha256)
            };

            var token = tokenHandler.CreateToken(tokenDescriptor);
            return tokenHandler.WriteToken(token);
        }
    }
}
```

---

3. Add Controllers

🔐 AuthController – Login API

📁 Controllers/AuthController.cs

```
using JwtAuthAPI.Models;
using JwtAuthAPI.Services;
using Microsoft.AspNetCore.Mvc;

namespace JwtAuthAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class AuthController : ControllerBase
    {
        private readonly JwtService _jwtService;

        public AuthController(JwtService jwtService)
        {
            _jwtService = jwtService;
        }
```

```
        [HttpPost("login")]
        public IActionResult Login([FromBody] LoginRequest request)
        {
                if (request.Username == "admin" && request.Password ==
"password")
            {
                                                        var    token    =
_jwtService.GenerateToken(request.Username);
                 return Ok(new { Token = token });
            }

            return Unauthorized();
        }
    }
}
```

✅ SecureController – Protected Endpoint

📁 Controllers/SecureController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace JwtAuthAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class SecureController : ControllerBase
    {
        [Authorize]
        [HttpGet("secure-data")]
        public IActionResult GetSecureData()
        {
            var username = User.Identity?.Name ?? "Unknown";
            return Ok($"This is secured data for {username}");
        }
```

```
    }
}
```

---

4. Update `Program.cs`

Minimal adjustments now that controllers handle endpoints.

```
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;
using JwtAuthAPI.Models;
using JwtAuthAPI.Services;
using Microsoft.OpenApi.Models;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddSingleton<JwtService>();
// Controllers
builder.Services.AddControllers();
//JWT Authentication
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationSch
eme)
    .AddJwtBearer(options =>
    {
                                                    var      key      =
Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]);
                            options.TokenValidationParameters    =    new
TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = builder.Configuration["Jwt:Issuer"],
            ValidAudience = builder.Configuration["Jwt:Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(key)
```

```
        };
    });

builder.Services.AddAuthorization();

// Enable Swagger with JWT support
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen(options =>
{
    options.AddSecurityDefinition("Bearer", new OpenApiSecurityScheme
    {
        Name = "Authorization",
        Type = SecuritySchemeType.ApiKey,
        Scheme = "Bearer",
        BearerFormat = "JWT",
        In = ParameterLocation.Header,
         Description = "Enter 'Bearer' followed by a space and your JWT
token."
    });

    options.AddSecurityRequirement(new OpenApiSecurityRequirement
    {
        {
            new OpenApiSecurityScheme
            {
                Reference = new OpenApiReference
                {
                    Type = ReferenceType.SecurityScheme,
                    Id = "Bearer"
                }
            },
            Array.Empty<string>()
        }
    });
});

var app = builder.Build();
```

```
// Middleware pipeline
// Enable Swagger UI

if (app.Environment.IsDevelopment())
{
    app.UseDeveloperExceptionPage();
    app.UseSwagger();
    app.UseSwaggerUI();

}
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

---

5. appsettings.json (No changes required)

Just ensure this section exists:

```
"Jwt": {
  "Key": "your-strong-key-goes-here",
  "Issuer": "https://localhost:5000",
  "Audience": "https://localhost:5000",
  "ExpiryInMinutes": 60
}
```

✅ Summary for Your Training Document

Layer	Description
`Models/LoginRequest.cs`	Captures login data
`Services/JwtService.cs`	Encapsulates logic to generate JWT

`Controllers/AuthController.cs`	Exposes `/api/auth/login` for token generation
`Controllers/SecureController.cs`	Protected API endpoint requiring JWT
`Program.cs`	Configures DI, Auth, Swagger, maps controllers
`appsettings.json`	Holds JWT config parameters

🧪 Testing via Swagger:

1. Run the app.
2. Go to Swagger UI.
3. Use `/api/auth/login` to get JWT token.
4. Click “Authorize” (padlock icon), paste: `Bearer <your_token>`.
5. Call `/api/secure/secure-data` — it should return personalized data.